

Assignment 03
Simple convolutional neural network to perform classification

Pattern Recognition
EN3150

Department of Electronic and Telecommunication Engineering
University of Moratuwa

Team Name : Opcode

Name	Index
Karunanayake A.H.D.	230321P
Piyumal N.P.P.	230493R
Shehan M.N.N.	230613M
Withanage G.D.N.	230735M

Date: October 19, 2026

Contents

1 Data Preparation **2**

1.1 Dataset and classes 2

1.2 Image preprocessing and augmentation 2

2 Custom Architecture Design **4**

2.1 Network configurations 4

2.2 Trainable-parameter calculation 4

2.3 Hardware-aware design choices 5

3 Optimizer Selection and Tuning **5**

3.1 Training configuration 5

4 Custom Model Training and Evaluation **5**

4.1 Training protocol 5

4.2 Training and validation curves 6

4.3 Test set classification performance 7

4.4 Confusion matrices 8

4.5 Resource utilization and A–B trade-offs 9

5 Lightweight SOTA Fine-Tuning and Evaluation **9**

6 Final Comparison and Trade-Off Discussion **11**

6.1 Accuracy Comparison 12

6.2 Memory Footprint 12

6.3 Computational Cost 12

6.4 Advantages and Limitations 13

6.5 Overall Conclusion 13

7 Source Code **13**

1 Data Preparation

1.1 Dataset and classes

The Intel Image Classification dataset (Puneet Chawla, Kaggle dataset. https://www.kaggle.com/datasets/puneet6060/intel-image-classification.) was used for this assignment. The `seg_train` and `seg_test` image directories were combined and split afresh, per class, into training, validation and testing subsets. The six scene categories are *buildings*, *forest*, *glacier*, *mountain*, *sea* and *street*.

Class	Training	Validation	Testing	Total
Buildings	1,839	394	395	2,628
Forest	1,921	411	413	2,745
Glacier	2,069	443	445	2,957
Mountain	2,125	455	457	3,037
Sea	1,948	417	419	2,784
Street	2,018	432	433	2,883
Total	**11,920**	**2,552**	**2,562**	**17,034**

Table 1: Stratified dataset split (seed = 42).

The intended proportions were 70%, 15% and 15%, respectively. The split manifest, `results/split.csv`, was reused across all eight experiments to prevent different test sets from biasing the comparisons.

1.2 Image preprocessing and augmentation

Input JPEGs were decoded as three-channel RGB images, resized to 64×64, and normalized to $[0, 1]$ by dividing each pixel value by 255. Training used a batch size of 32 and shuffled examples with random seed 42. Validation and test inputs were not shuffled or augmented. The training only augmentation consisted of random horizontal flipping, rotation and zooming.

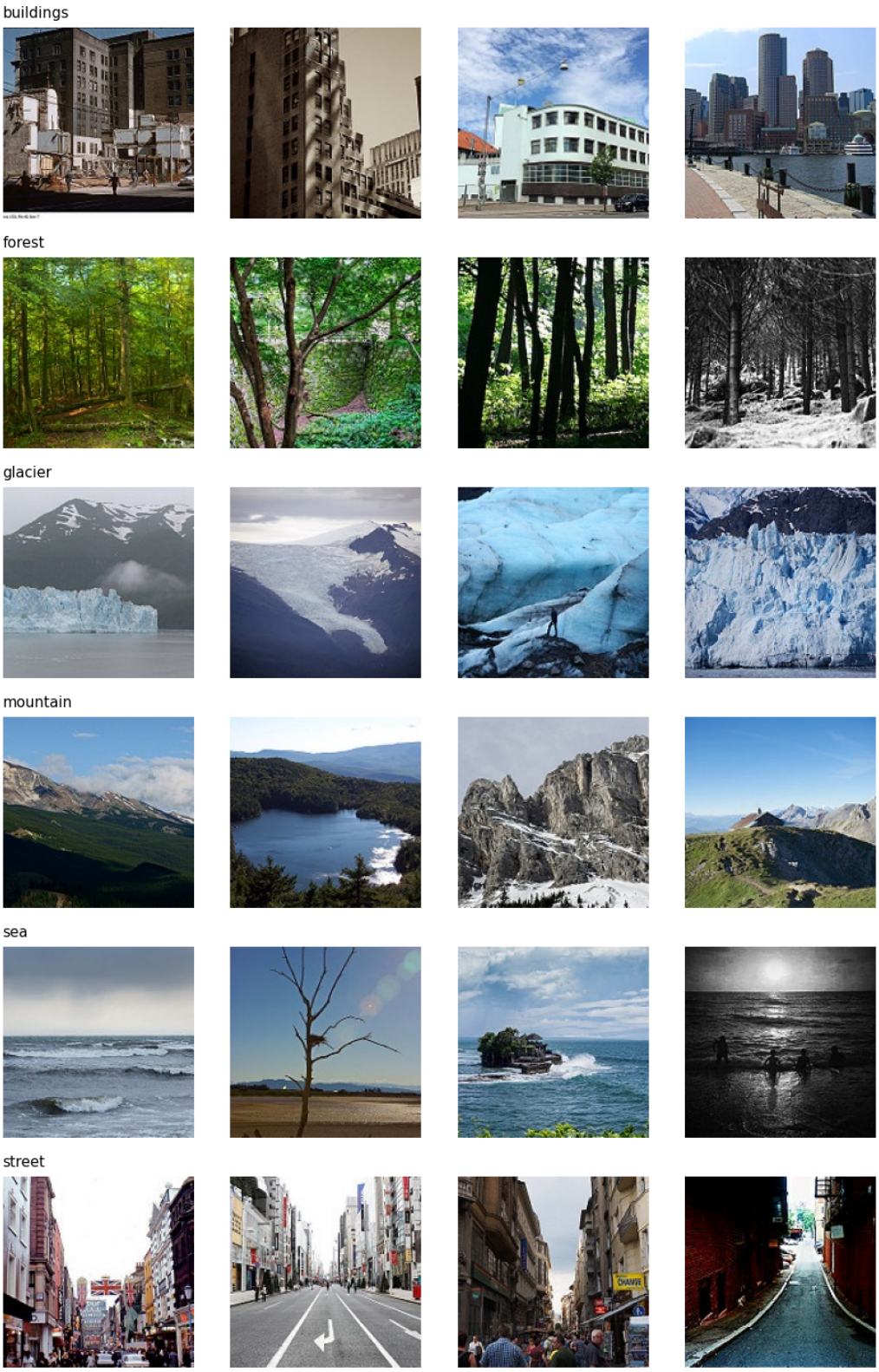


Figure 1: Representative images from the six scene categories.

2 Custom Architecture Design

2.1 Network configurations

Both networks accept $64 \times 64 \times 3$ input tensors and contain four convolution, normalization, activation and pooling blocks. Model A uses conventional 3×3 convolutions, while Model B replaces these with 3×3 depthwise convolutions followed by 1×1 pointwise convolutions. All convolutions use same padding. Each block uses batch normalization with momentum 0.9, ReLU and 2×2 max pooling. The block filter counts are 32, 64, 128 and 128, respectively.

Stage	**Filters / units**	**Kernel**	**Output shape**
Input	—	—	$64 \times 64 \times 3$
Block 1	32	3×3	$32 \times 32 \times 32$
Block 2	64	3×3	$16 \times 16 \times 64$
Block 3	128	3×3	$8 \times 8 \times 128$
Block 4	128	3×3	$4 \times 4 \times 128$
Global average pooling	—	—	128
Dropout	A: 0.30; B: 0.20	—	128
Dense + ReLU	64	—	64
Dense + softmax	6	—	6 classes

Table 2: Layer structure of the custom CNNs (output is after pooling).

Global average pooling was chosen instead of flattening the $4 \times 4 \times 128$ feature map. It produces only 128 features, reducing the size of the final fully connected classifier. The final softmax layer returns probabilities for six mutually exclusive classes.

2.2 Trainable-parameter calculation

For a conventional convolution with kernel width k, input channels C_{in} and output channels C_{out}, including biases,

$$P_{\text{standard}} = k^2 C_{\text{in}} C_{\text{out}} + C_{\text{out}}. \tag{1}$$

For the implemented depthwise separable convolution (without biases),

$$P_{\text{separable}} = k^2 C_{\text{in}} + C_{\text{in}} C_{\text{out}}. \tag{2}$$

Each batch normalization layer introduces $2C_{\text{out}}$ trainable scale and shift parameters, as well as $2C_{\text{out}}$ non-trainable running statistics. A dense layer with n_{in} inputs and n_{out} outputs has $n_{\text{in}} n_{\text{out}} + n_{\text{out}}$ trainable parameters.

Component	**Model A**	**Model B**
Block 1 (including BatchNorm)	960	187
Block 2 (including BatchNorm)	18,624	2,464
Block 3 (including BatchNorm)	74,112	9,024
Block 4 (including BatchNorm)	147,840	17,792
Dense $128 \to 64$	8,256	8,256
Dense $64 \to 6$	390	390
Trainable parameters	**250,182**	**38,113**
Non-trainable BatchNorm statistics	704	704
Total parameters	**250,886**	**38,817**

Table 3: Calculated trainable parameters of the custom models.

2.3 Hardware-aware design choices

ReLU uses $\max(0, x)$ and avoids the exponential evaluations required by sigmoid/tanh activation in hidden layers. Max pooling lowers feature map resolution after each block, reducing work in deeper layers. Global average pooling and separable convolution further reduce parameter storage and MACs. Batch normalization can also be folded into adjacent convolution weights for some inference deployment pipelines. Softmax is retained only at the six class output, where normalized class probabilities are useful.

Ignoring normalization, pooling, activation and preprocessing overhead, the estimated multiply accumulate count for these implemented networks is approximately 50,733,440 MACs/image (A) and 6,271,360 MACs/image (B) or about an 8.1× reduction.

3 Optimizer Selection and Tuning

3.1 Training configuration

Adam was selected as the principal optimizer for comparing A and B because its adaptive step sizes provide a practical baseline for training the two networks. For context, standard SGD and SGD with momentum were also tested for each architecture. The learning rates used in the code are given in Table 4. Momentum was set to $\mu = 0.9$.

Model	Optimizer	Initial LR	Best epoch	Best val. loss	Val. accuracy
A	Adam	0.001	18	0.3836266100406647	0.8644200563430786
A	SGD	0.005	13	0.6130951642990112	0.7719435691833496
A	SGD + momentum	0.005	18	0.41733309626579285	0.8561912178993225
B	Adam	0.001	20	0.47734954953193665	0.8244513869285583
B	SGD	0.010	20	0.5912036895751953	0.7872257232666016
B	SGD + momentum	0.010	18	0.53400719165802	0.8013322949409485

Table 4: Optimizer comparison.

Observed optimizer behavior: The results in Table 4 show that Adam achieved the best validation performance for both CNN architectures. For Model A, Adam obtained the lowest validation loss of 0.3836 with a corresponding validation accuracy of 86.44%, while Model B achieved a validation loss of 0.4773 and an accuracy of 82.45%. Standard SGD showed the weakest performance among the three optimizers for both architectures. Introducing momentum improved the validation accuracy of Model A from 77.19% to 85.62%, an increase of approximately 8.43 percentage points. Similarly, Model B showed an improvement from 78.72% to 80.13%, representing an increase of approximately 1.41 percentage points. The validation loss also decreased with momentum for both architectures. Since standard SGD and SGD with momentum used the same initial learning rate within each model, these results provide stronger evidence that momentum improved the optimization process under the tested conditions. However, Adam still achieved the lowest validation loss and highest validation accuracy for both models. Therefore, Adam was selected as the preferred optimizer for subsequent evaluation of the custom CNN architectures.

4 Custom Model Training and Evaluation

4.1 Training protocol

Each custom model was trained with sparse categorical cross-entropy for 20 epochs. A `ReduceLROnPlateau` callback halved the learning rate after two epochs without improvement in validation loss (minimum

LR 10^{-5}). The best validation loss checkpoint was saved for test evaluation. An `EarlyStopping` callback with patience five can stop longer runs, but is configured not to begin monitoring for early stopping before epoch 20. Augmentation was applied to training inputs only. Validation and test images were not augmented. The selected Adam checkpoints are used for the detailed test analysis below.

4.2 Training and validation curves

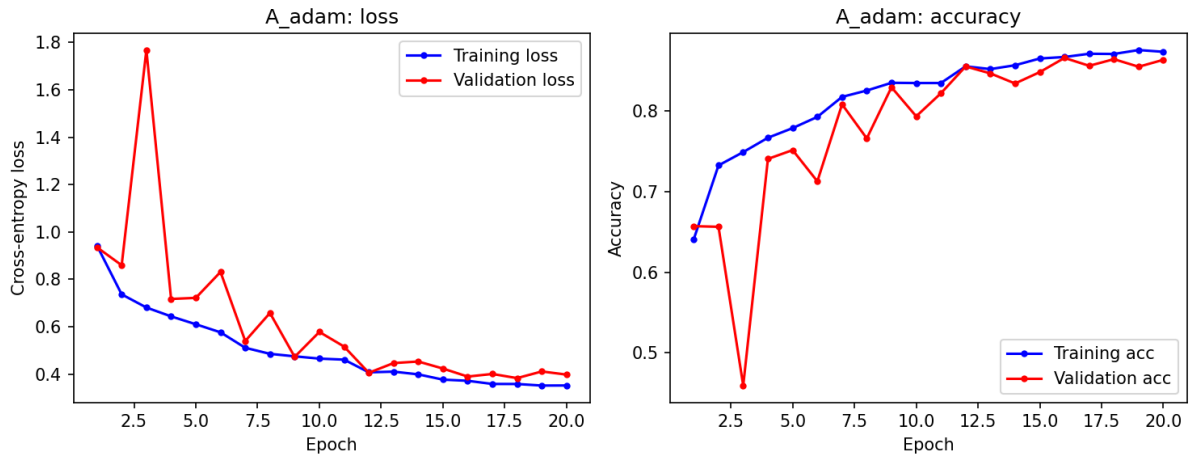


Figure 2: Model A with Adam: training and validation loss/accuracy.

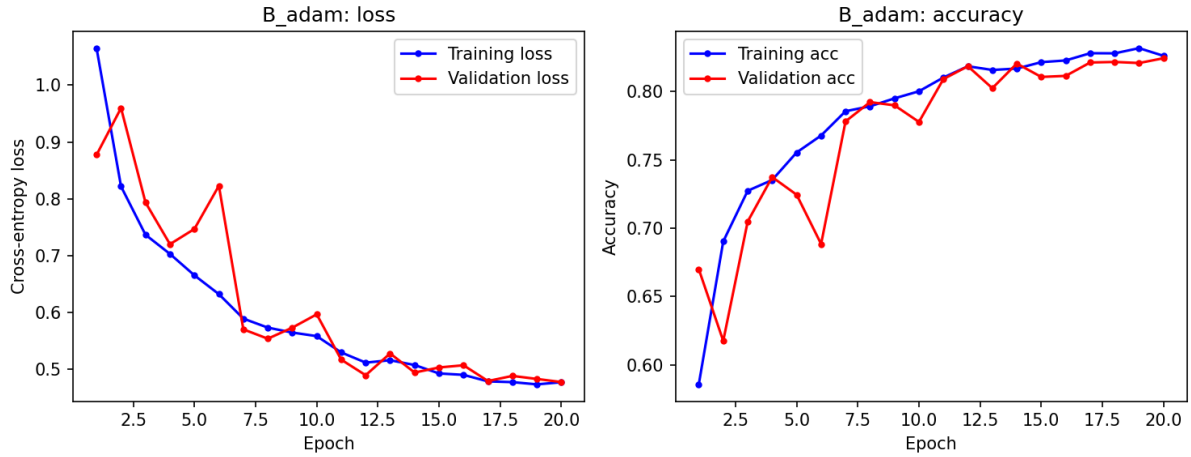


Figure 3: Model B with Adam: training and validation loss/accuracy.

Interpretation: As shown in Figures 2 and 3, both Model A and Model B demonstrate successful learning, with training and validation losses generally decreasing while accuracy improves over the 20 epochs. Model A exhibits noticeable fluctuations in validation loss during the initial epochs, including a significant spike around epoch 3. However, these fluctuations gradually decrease and the validation loss becomes more stable after approximately epoch 12. Model B also shows some early fluctuations, but its validation loss follows a relatively smoother downward trend after approximately epoch 7.

Model A achieved its minimum validation loss at epoch 18, while Model B achieved its minimum at epoch 20. Towards the final epochs, the training and validation loss curves remain close to each other for both models, indicating no clear evidence of significant overfitting. The accuracy curves also show that both models achieve similar training and validation accuracies towards the end of

training. Model A generally achieves higher accuracy and lower loss than Model B, suggesting better classification performance. However, Model B provides a more parameter efficient architecture, making it potentially more suitable for resource constrained applications.

4.3 Test set classification performance

The final selected checkpoints were evaluated once on the same 2,562 image test subset. The metrics are test accuracy, per-class precision, per-class recall and macro-averaged precision/recall.

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}. \tag{3}$$

Macro averages are the unweighted means across all six classes.

Model	**Test accuracy**	**Macro precision**	**Macro recall**
A	0.8715846994535519	0.8726121239829081	0.8735266100015298
B	0.8224043715846995	0.8251513249079981	0.8249725628067507

Table 5: Custom models results using the selected Adam checkpoints

Class	**A precision**	**A recall**	**B precision**	**B recall**
Buildings	0.8639798488664987	0.8683544303797468	0.7270788912579957	0.8632911392405064
Forest	0.9636803874092009	0.9636803874092009	0.9466019417475728	0.9443099273607748
Glacier	0.8313817330210773	0.797752808988764	0.8158567774936062	0.7168539325842697
Mountain	0.8265765765765766	0.8030634573304157	0.7757847533632287	0.7571115973741794
Sea	0.8493449781659389	0.9284009546539379	0.85	0.8114558472553699
Street	0.900709219858156	0.8799076212471132	0.8355855855855856	0.8568129330254042

Table 6: Per-class test precision and recall

4.4 Confusion matrices

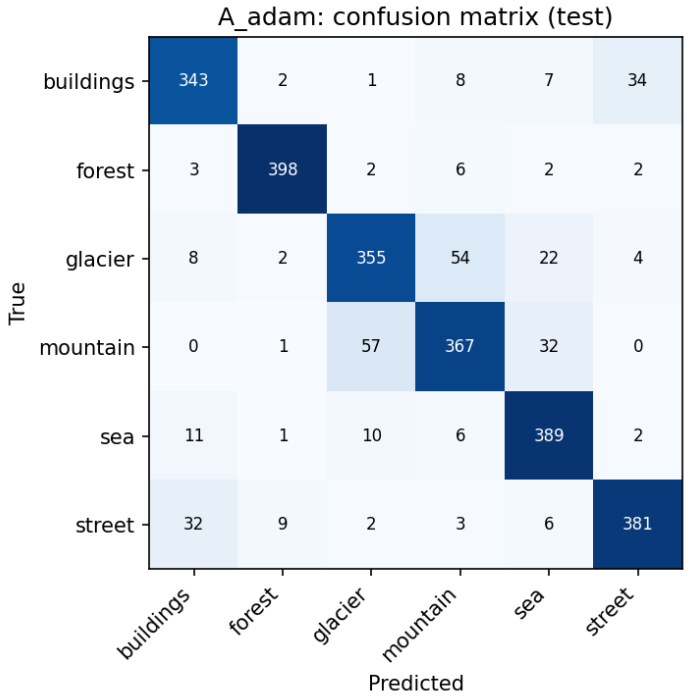


Figure 4: Test-set confusion matrix for Model A with Adam.

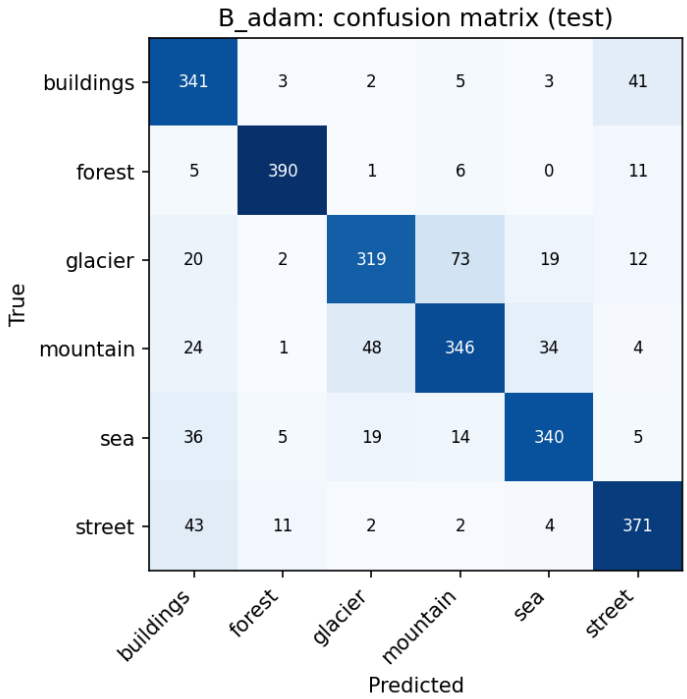


Figure 5: Test-set confusion matrix for Model B with Adam.

Error analysis: Figures 4 and 5 show that both models correctly classify most test images, as indicated by the high values along the main diagonal of the confusion matrices. However, some misclassifications occur between visually similar scene categories. The most significant confusion is observed between glacier and mountain images. Model A incorrectly classified 54 glacier images as

mountain and 57 mountain images as glacier. Model B made 73 and 48 errors in these respective categories. This confusion may be attributed to the similar visual characteristics of these scenes, such as mountainous landscapes, snow, rocks and overlapping natural textures.

Another noticeable confusion occurs between buildings and street images. Model A misclassified 34 building images as street and 32 street images as buildings, while Model B made 41 and 43 such errors, respectively. This may be due to the presence of buildings, roads and other urban structures in both categories. Furthermore, Model B shows more classification errors involving sea images, particularly misclassifying 36 sea images as buildings compared with 11 for Model A.

Overall, Model A demonstrates better classification performance, with fewer total misclassifications than Model B. The lightweight architecture of Model B reduces the number of trainable parameters and computational requirements, but this comes with some reduction in its ability to distinguish visually similar scenes. But overall, both models demonstrate reasonable classification performance across the six categories.

4.5 Resource utilization and A–B trade-offs

Measure	**Model A**	**Model B**
Total parameters	250,886	38,817
Trainable parameters	250,182	38,113
MACs/image (estimate)	50,733,440	6,271,360
Float32 weight estimate (KiB)	980.0	151.6
Saved `.keras` file (KiB)	3033.08	549.68
Test accuracy	0.8716	0.8224

Table 7: Resource and accuracy comparison of the selected custom CNNs

The float32 estimates use four bytes per parameter and 1 KiB = 1024 bytes. The actual `.keras` files can be larger because they contain metadata and may contain optimizer state. The mean custom model epoch times are measured from epochs 2 onward, excluding first epoch dataset cache initialization. Model B has approximately 84.5% fewer total parameters and 87.6% fewer estimated convolution/dense MACs than Model A.

Measured trade-off: As shown in Table 7, Model B achieves a significant reduction in resource requirements compared with Model A, with approximately 84.5% fewer parameters and 87.6% fewer estimated MAC operations per image. The saved model size is also reduced from 3033.08 KiB to 549.68 KiB, representing a reduction of approximately 81.9%. However, this efficiency comes at the cost of a slight reduction in classification accuracy. Model A achieved a test accuracy of 87.16%, whereas Model B achieved 82.24%, resulting in a difference of 4.92 percentage points.

Overall, Model A provides better classification accuracy, while Model B offers substantial savings in memory usage and theoretical computational cost. For resource constrained edge devices where memory and computational capacity are limited, the efficiency improvements of Model B may justify the moderate reduction in accuracy.

5 Lightweight SOTA Fine-Tuning and Evaluation

The two lightweight reference models are MobileNetV2 and EfficientNet-B0, both loaded with ImageNet-pretrained weights and a new head consisting of global average pooling, dropout 0.2 and a six unit softmax output. They use the same fixed dataset splits as A and B. Since the loader emits $[0, 1]$ pixels, MobileNetV2 receives $2x - 1$ inputs, EfficientNet-B0 receives $255x$ inputs and applies its internal normalization.

Both reference models used a two stage procedure:

- **Stage 1:** Freeze the backbone and train the classification head for five epochs with Adam ($\eta = 10^{-3}$).
- **Stage 2:** Unfreeze the complete backbone, recompile and fine tune for fifteen epochs with Adam ($\eta = 10^{-4}$).

The best validation loss weights saved during stage 2 were loaded for final testing.

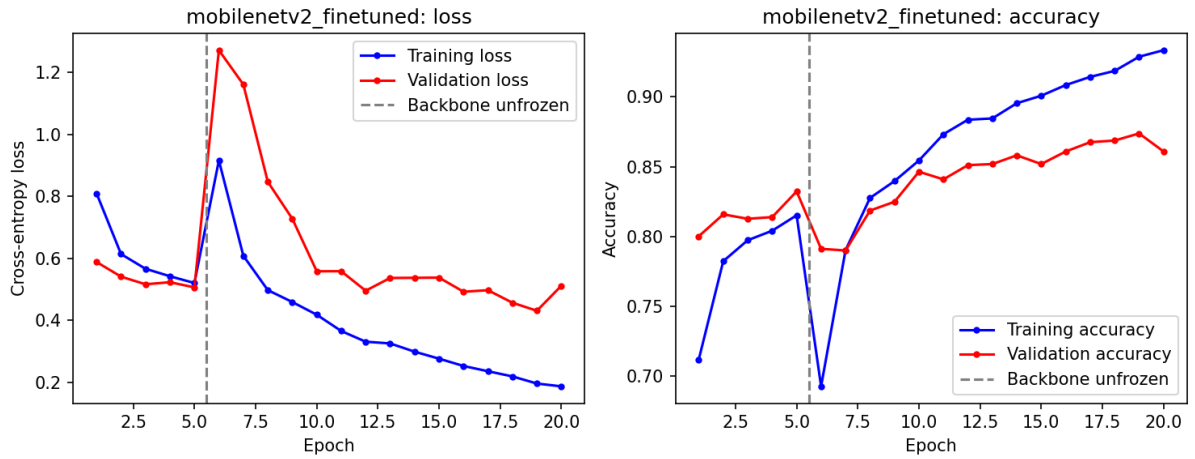


Figure 6: MobileNetV2: loss/accuracy for frozen-head training (epochs 1–5) and full fine-tuning (epochs 6–20).

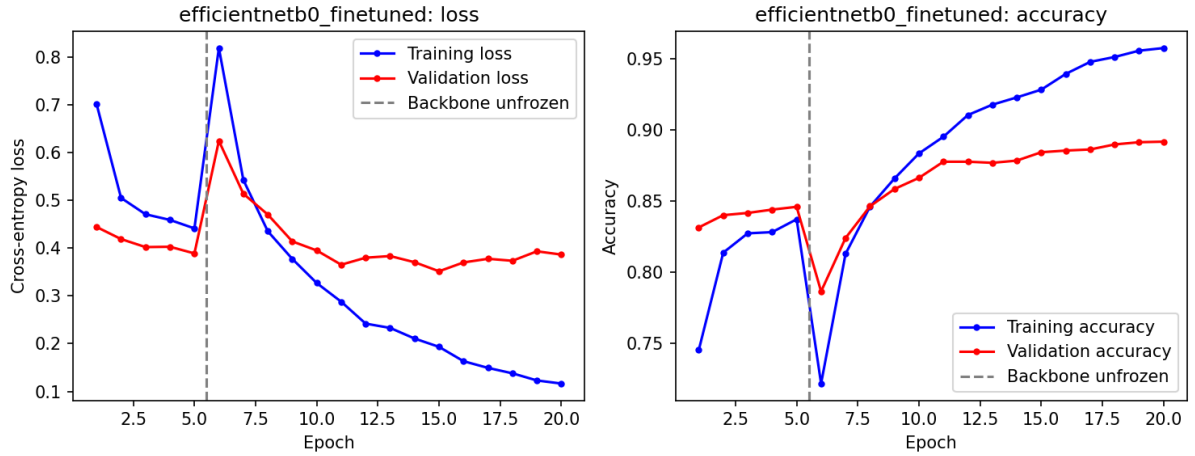


Figure 7: EfficientNet-B0: loss/accuracy for frozen-head training (epochs 1–5) and full fine-tuning (epochs 6–20).

Table 8: Fine-tuned lightweight pre-trained models: final test performance.

Measure	**MobileNetV2**	**EfficientNet-B0**
Total parameters	2,265,670	4,057,257
Float32 weight estimate (MiB)	8.64	15.48
Saved weights file (MiB)	26.20	46.86
Mean stage-2 training time/epoch (s)	460.02	576.71
Test accuracy	0.8758782201405152	0.8911007025761124
Macro precision	0.8791753204688945	0.8924198712261239
Macro recall	0.8778102445713546	0.8932699490925095

The float32 estimates above are parameter storage estimates rather than full deployment model sizes. Saved weight files can also include additional state/metadata. The reported mean epoch time here comes from the fine-tuning stage, not the frozen-backbone stage.

Fine-tuning interpretation: Figures 6 and 7 illustrate the training and validation performance of MobileNetV2 and EfficientNet-B0 during the two-stage transfer learning process. During the first five epochs, only the classification head was trained while the pre-trained backbone remained frozen. Both models demonstrated improvements in training and validation performance during this stage.

At epoch 6, when the backbones were unfrozen for fine tuning, both models exhibited a noticeable increase in training and validation loss, accompanied by a temporary decrease in accuracy. This behavior may be attributed to the adjustment of pre-trained features when the backbone parameters became trainable. Following this transition, the losses decreased and the accuracies improved, indicating that the models were able to adapt to the target dataset.

However, towards the later epochs, a noticeable gap developed between the training and validation curves. For MobileNetV2, training loss continued to decrease, while validation loss fluctuated and remained relatively stable. A similar pattern was observed for EfficientNet-B0, where training accuracy continued to improve while validation accuracy showed only limited improvement. These observations suggest some degree of overfitting during fine-tuning, particularly towards the final epochs.

According to Table 8, EfficientNet-B0 achieved a higher test accuracy of 89.11% compared with 87.59% for MobileNetV2. EfficientNet-B0 also obtained higher macro precision and recall, indicating better overall classification performance across the six classes. However, EfficientNet-B0 required more parameters and a larger memory footprint. Its recorded average training time per epoch was also considerably higher, although differences in system conditions may have influenced the timing measurements.

Overall, EfficientNet-B0 provided better classification performance, while MobileNetV2 offered a more compact alternative with slightly lower accuracy. These results highlight the trade-off between classification performance and resource requirements when selecting pre-trained architectures for edge applications.

6 Final Comparison and Trade-Off Discussion

Table 9 summarizes the performance and resource requirements of the four evaluated CNN architectures. The comparison considers classification accuracy, model parameters, memory footprint and computational requirements to evaluate their suitability for resource constrained edge devices.

Model	Parameters	Float32 MiB	Test accuracy	Macro recall
Model A	250,886	0.957	87.16%	87.35%
Model B	38,817	0.148	82.24%	82.50%
MobileNetV2	2,265,670	8.64	87.59%	87.78%
EfficientNet-B0	4,057,257	15.48	89.11%	89.33%

Table 9: Final performance and resource comparison of the custom and pre-trained CNN architectures.

6.1 Accuracy Comparison

As shown in Table 9, EfficientNet-B0 achieved the highest test accuracy of 89.11%, followed by MobileNetV2 with 87.59%. Among the custom architectures, Model A achieved 87.16%, while the lightweight Model B obtained 82.24%.

Compared with Model B, Model A improved test accuracy by 4.92 percentage points. MobileNetV2 and EfficientNet-B0 achieved improvements of 5.35 and 6.87 percentage points, respectively.

An interesting observation is that Model A achieved a test accuracy only 0.43 percentage points lower than MobileNetV2, despite having approximately 88.9% fewer parameters. This indicates that the custom standard CNN provides competitive classification performance with a considerably smaller parameter count.

The higher accuracy of the pre-trained networks suggests that transfer learning was beneficial for this scene classification task. However, the relatively small accuracy improvements must be considered alongside their substantially greater memory requirements.

6.2 Memory Footprint

Model B has the smallest memory footprint, requiring approximately 0.148 MiB for its Float32 parameters. Model A requires approximately 0.957 MiB, while MobileNetV2 and EfficientNet-B0 require 8.64 MiB and 15.48 MiB, respectively.

Model B contains approximately 84.5% fewer parameters than Model A. Compared with MobileNetV2 and EfficientNet-B0, Model B uses approximately 98.3% and 99.0% fewer parameters, respectively.

These results demonstrate the advantage of designing a custom lightweight architecture for devices with limited memory capacity.

However, these Float32 estimates only represent parameter storage. Actual memory consumption during inference also includes intermediate activations, runtime buffers and framework overhead. Therefore, the complete memory requirements must be evaluated on the target hardware before deployment.

6.3 Computational Cost

The estimated computational requirements of Model A and Model B are approximately 50.73 million and 6.27 million multiply-accumulate operations (MACs) per image, respectively.

Replacing standard convolutions with depthwise separable convolutions reduced the estimated MAC count by approximately 87.6%. This makes Model B theoretically more computationally efficient than Model A.

Although MobileNetV2 and EfficientNet-B0 are designed for efficient mobile and edge inference, their computational requirements were not measured using the same MAC-counting procedure in this experiment. Therefore, a direct quantitative comparison of inference MACs across all four models cannot be made.

Furthermore, CPU training time is not a reliable measure of inference efficiency, particularly because the models were trained during separate sessions under potentially different system conditions.

Actual inference latency and energy consumption would need to be benchmarked under identical conditions on the intended deployment hardware.

6.4 Advantages and Limitations

Model A achieved better classification accuracy than Model B while maintaining a substantially smaller parameter count than the pre-trained architectures. It therefore represents a compromise between classification performance and model complexity.

Model B provides the greatest reduction in parameter storage and estimated computational cost. However, its limited model capacity may reduce its ability to distinguish visually similar scene categories, as observed in the confusion matrix analysis.

MobileNetV2 offers higher classification accuracy than both custom models while using fewer parameters than EfficientNet-B0. This makes it a potentially suitable option for edge devices with moderate memory and processing capabilities.

EfficientNet-B0 achieved the highest classification accuracy, but it also has the largest parameter count and Float32 weight-storage requirement among the evaluated architectures.

Further optimizations, such as post-training quantization and hardware-specific inference runtimes, could improve deployment efficiency. However, their effects on accuracy, memory consumption, and inference speed would need to be evaluated separately.

6.5 Overall Conclusion

The experimental results demonstrate a clear trade-off between classification accuracy and resource requirements.

For highly resource constrained devices, Model B is the preferred candidate because it requires only 38,817 parameters and approximately 6.27 million estimated MACs per image while achieving 82.24% test accuracy.

If the deployment hardware can accommodate a larger network and classification accuracy is the primary requirement, EfficientNet-B0 would be preferable, achieving the highest test accuracy of 89.11%.

Model A offers an intermediate solution between the compact Model B and the pre-trained networks, providing improved accuracy with a relatively small parameter count.

Overall, the most suitable architecture depends on the available memory, processing capability, energy budget, and acceptable classification accuracy. Final deployment decisions should also consider measurements of inference latency and memory consumption on the actual target device.

7 Source Code

The Python/TensorFlow implementation uses seed 42 and the same `results/split.csv` manifest for all model families. Custom-model training is implemented in `src/train.py`, transfer learning in `src/sota.py` and evaluation in `src/evaluate.py`. Training histories, measurements, checkpoints and plots are stored under `results/`, `models/` and `plots/` respectively. The repository and instructions are available at: https://github.com/HirunaKarunanayake/edge-image-classification.